

SQL: Data Definition

Lecture 06

Borough of Manhattan Community College

SQL Identifiers

- ❑ SQL identifiers are used to identify objects in the database, such as table names, view names, and columns.
- ❑ The ISO standard provides a default character set for SQL identifier, which consists of:
 - the uppercase letters A . . . Z;
 - the lowercase letters a . . . z;
 - the digits 0 . . . 9;
 - the underscore (_) character.
- ❑ The following restrictions are imposed on an identifier:
 - an identifier can be no longer than 128 characters;
 - an identifier must start with a letter;

Character Data Type

- ❑ The format for specifying a character data type is:

CHARACTER [VARYING] [length]

- ❑ The length can be specified to indicate the maximum number of characters that the column can hold (default length is 1).
- ❑ A character string may be defined as having a fixed or varying length.
- ❑ **CHARACTER** can be abbreviated to **CHAR**.
- ❑ **CHARACTER VARYING** can be abbreviated to **VARCHAR**.

Character Data Type

- ❑ If the string is defined to be a fixed length and we enter a string with fewer characters than this length, the string is padded with blanks on the right to make up the required size.

branchNo **CHAR**(4)

- ❑ If the string is defined to be of a varying length and we enter a string with fewer characters than this length, only those characters entered are stored, thereby using less space.

address **VARCHAR**(30)

Numeric Data Type

- ❑ This data type is used to define numbers.

A number consists of digits, an optional decimal point, and an optional sign.

- ❑ A numeric data type consists of a precision and a scale.
 - The precision gives the total number of significant decimal digits, that is, the total number of digits including decimal places.
 - The scale gives the total number of decimal places.
 - For example, the numeric value -62.315 has precision 5 and scale 3.
- ❑ There are several ways of specifying a numeric data type:
 - **DECIMAL** [precision [, scale]]
 - **INTEGER**
 - **SMALLINT**
 - **BIGINT**
 - **FLOAT** [precision]

Numeric Data Type

- ❑ The column *rooms* of the *PropertyForRent* table, which represents the number of rooms in a property, is obviously a small integer and can be declared as:

rooms **SMALLINT**

- ❑ The column *salary* of the *Staff* table can be declared as:

salary **DECIMAL(7,2)**

which can handle a value up to 99,999.99

Datetime Data Type

- ❑ The datetime data type is used to define points in time to a certain degree of accuracy.
- ❑ Three types of datetime data type are supported:
 - **DATE** : Used to store calendar dates using the YEAR, MONTH, and DAY fields.
 - **TIME** [timePrecision] : Used to store time using the HOUR, MINUTE, and SECOND fields.
 - **TIMESTAMP** [timePrecision] : Used to store date and time.
- ❑ The *timePrecision* is the number of decimal places of accuracy to which the SECOND field is kept.
- ❑ For example, the column ***viewDate*** of the ***Viewing*** table, which represents the date that a client viewed a property, is declared as:

viewDate **DATE**

Scalar Operators

- ❑ SQL provides a number of built-in scalar operators and functions that can be used to construct a scalar expression, that is, an expression that evaluates to a scalar value.
- ❑ Apart from the obvious arithmetic operators (+, −, *, /), the following operators are available:

OPERATOR	MEANING
CHAR_LENGTH	Returns the length of a string in characters (or octets, if the string is a bit string). For example, CHAR_LENGTH ('Beech') returns 5.
CAST	Converts a value expression of one data type into a value in another data type. For example, CAST (5.2E6 AS INTEGER).
LOWER	Converts uppercase letters to lowercase. For example, LOWER (SELECT fName FROM Staff WHERE staffNo = 'SL21') returns 'john'.
UPPER	Converts lower-case letters to upper-case. For example, UPPER (SELECT fName FROM Staff WHERE staffNo = 'SL21') returns 'JOHN'.
 	Concatenates two character strings or bit strings. For example, fName IName.

OPERATOR	MEANING
TRIM	Removes leading (LEADING), trailing (TRAILING), or both leading and trailing (BOTH) characters from a string. For example, TRIM(BOTH '*' FROM '*** Hello World ***) returns 'Hello World'.
POSITION	Returns the position of one string within another string. For example, POSITION('ee' IN 'Beech') returns 2.
SUBSTRING	Returns a substring selected from within a string. For example, SUBSTRING('Beech' FROM 1 TO 3) returns the string 'Bee'.
CURRENT_DATE	Returns the current date in the time zone that is local to the user.
CURRENT_TIME	Returns the current time in the time zone that is the current default for the session. For example, CURRENT_TIME(6) gives time to microseconds precision.
CURRENT_TIMESTAMP	Returns the current date and time in the time zone that is the current default for the session. For example, CURRENT_TIMESTAMP(0) gives time to seconds precision.
POWER	Raises its first argument to the power of its second argument. For example, POWER(2,3) returns 8.
SQRT	Computes the square root of its argument. For example, SQRT(16) returns 4.
FLOOR	Computes the greatest integer less than or equal to its argument. For example, FLOOR(15.7) returns 15.
CEIL	Computes the least integer greater than or equal to its argument. For example, CEIL(15.7) returns 16.
ABS	Operates on a numeric argument and returns its absolute value in the same most specific type. For example, ABS(-17.1) returns 17.1.

Integrity Enhancement Feature

- ❑ Integrity control consists of constraints that we wish to impose in order to protect the database from becoming inconsistent.

- ❑ Five types of integrity constraint are considered:
 - required data;
 - domain constraints;
 - entity integrity;
 - referential integrity;
 - general constraints.

- ❑ These constraints can be defined in the CREATE and ALTER TABLE statements.

Required Data

- ❑ Some columns must contain a valid value (nulls are not allowed).
- ❑ For example, every member of staff must have an associated job position (Manager, Assistant, and so on).
- ❑ The ISO standard provides the **NOT NULL** column specifier in the CREATE and ALTER TABLE statements to provide this type of constraint.
- ❑ For example, to specify that the column *position* of the *Staff* table cannot be null, we define the column as:
position **VARCHAR(10) NOT NULL**
- ❑ When NOT NULL is specified, the system rejects any attempt to insert a null in the column.

Domain Constraints

- ❑ Every column has a domain (a set of legal values).
- ❑ For example, the **sex** of a member of **staff** is either 'M' or 'F'.
- ❑ The ISO standard provides mechanisms for specifying domains in the CREATE and ALTER TABLE statements.
- ❑ The **CHECK** clause allows a constraint to be defined on a column or the entire table. The format of the CHECK clause is:
CHECK (searchCondition)

- ❑ The **sex** column can be defined as:

sex **CHAR NOT NULL CHECK** (sex **IN** ('M', 'F'))

Domain Constraints

- ❑ The ISO standard allows domains to be defined more explicitly using the **CREATE DOMAIN** statement:

```
CREATE DOMAIN DomainName AS dataType  
[DEFAULT defaultOption]  
[CHECK (searchCondition)]
```

- ❑ We could define a domain for **sex** as:

```
CREATE DOMAIN SexType AS CHAR  
DEFAULT 'M'  
CHECK (VALUE IN ('M', 'F'));
```

- ❑ When defining the column **sex**, we can now use the domain name **SexType** in place of the data type **CHAR**:

```
sex SexType NOT NULL
```

Entity Integrity

- ❑ The primary key of a table must contain a unique, non-null value for each row.
- ❑ The ISO standard supports entity integrity with the PRIMARY KEY clause in the CREATE and ALTER TABLE statements.
- ❑ For example, to define the primary key of the *PropertyForRent* table, we include the following clause:

PRIMARY KEY(propertyNo)

- ❑ To define a composite primary key, we specify multiple column names in the PRIMARY KEY clause, separating each by a comma. For example:

PRIMARY KEY(clientNo, propertyNo)

Entity Integrity

- ❑ The PRIMARY KEY clause can be specified only once per table.
- ❑ It is possible to ensure uniqueness for any alternate keys in the table using the keyword **UNIQUE**.
- ❑ Every column that appears in a UNIQUE clause must also be declared as NOT NULL. For example:

```
clientNo VARCHAR(5) NOT NULL,  
propertyNo VARCHAR(5) NOT NULL,  
UNIQUE (clientNo, propertyNo)
```
- ❑ SQL rejects any INSERT or UPDATE operation that attempts to create a duplicate value within each candidate key.

Referential Integrity

- ❑ A foreign key is a column, or set of columns, that links each row in the child table (containing the foreign key) to the row of the parent table containing the matching candidate key value.
- ❑ For example, *branchNo* in the *Staff* table links the staff member to that row in the *Branch* table where the staff is working.
- ❑ The ISO standard supports the definition of foreign keys with the FOREIGN KEY clause in the CREATE and ALTER TABLE statements.
FOREIGN KEY(branchNo) **REFERENCES** Branch

Referential Integrity

Branch

branchNo	street	city	postcode
B005	22 Deer Rd	London	SW1 4EH
B007	16 Argyll St	Aberdeen	AB2 3SU
B003	163 Main St	Glasgow	G11 9QX
B004	32 Manse Rd	Bristol	BS99 1NZ
B002	56 Clover Dr	London	NW10 6EU

Primary key

Foreign key

Staff

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SL21	John	White	Manager	M	1-Oct-45	30000	B005
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000	B003
SA9	Mary	Howe	Assistant	F	19-Feb-70	9000	B007
SG5	Susan	Brand	Manager	F	3-Jun-40	24000	B003
SL41	Julie	Lee	Assistant	F	13-Jun-65	9000	B005

Referential Integrity

- ❑ The action SQL takes for any UPDATE or DELETE operation that attempts to update or delete a candidate key value in the parent table is dependent on the **referential action** specified in the FOREIGN KEY clause.

- ❑ When the user attempts to delete a row from a parent table, and there are one or more matching rows in the child table, SQL supports four options regarding the action to be taken:
 - CASCADE
 - SET NULL
 - SET DEFAULT
 - NO ACTION

Referential Integrity

- ❑ **CASCADE:** Delete the row from the parent table and automatically delete the matching rows in the child table.
- ❑ **SET NULL:** Delete the row from the parent table and set the foreign key value(s) in the child table to NULL.
- ❑ **SET DEFAULT:** Delete the row from the parent table and set each component of the foreign key in the child table to the specified default value.
- ❑ **NO ACTION:** Reject the delete operation from the parent table.
This is the default setting if the **referential action** is not specified.

Referential Integrity

- ❑ SQL supports the same options when the candidate key in the parent table is updated.
- ❑ With CASCADE, the foreign key value(s) in the child table are set to the new value(s) of the candidate key in the parent table.
- ❑ Examples from the *PropertyForRent* table:

FOREIGN KEY (staffNo) **REFERENCES** Staff **ON DELETE SET NULL**

FOREIGN KEY (ownerNo) **REFERENCES** PrivateOwner **ON UPDATE CASCADE**

General Constraints

- ❑ Updates to tables may be constrained by enterprise business rules.
For example, *DreamHome* may have a rule that prevents a member of staff from managing more than 100 properties at the same time.
- ❑ The ISO standard allows general constraints to be specified using the CHECK and UNIQUE clauses of the CREATE and ALTER TABLE statements and the CREATE ASSERTION statement.
- ❑ The CREATE ASSERTION statement is an integrity constraint that is not directly linked with a table definition.

General Constraints

- ❑ The format of the statement is:

```
CREATE ASSERTION AssertionName  
CHECK (searchCondition)
```

- ❑ For example, to define the general constraint that prevents a member of staff from managing more than 100 properties at the same time, we could write:

```
CREATE ASSERTION StaffNotHandlingTooMuch  
CHECK (NOT EXISTS (SELECT staffNo  
                     FROM PropertyForRent  
                     GROUP BY staffNo  
                     HAVING COUNT(*) > 100))
```

Creating a Table

- The CREATE TABLE statement has the following basic syntax:

```
CREATE TABLE TableName (  
    {columnName dataType [NOT NULL] [UNIQUE]  
    [DEFAULT defaultOption] [CHECK (searchCondition)] [, . . .]}  
    [PRIMARY KEY (listOfColumns),  
    {[UNIQUE (listOfColumns)] [, . . .]}  
    {[FOREIGN KEY (listOfForeignKeyColumns)  
    REFERENCES ParentTableName [(listOfCandidateKeyColumns)]  
        [ON UPDATE referentialAction]  
        [ON DELETE referentialAction]} [, . . .]}  
    {[CHECK (searchCondition)] [, . . .]}  
);
```

```

CREATE TABLE PropertyForRent(
    propertyNo    VARCHAR(5) NOT NULL,
    street        VARCHAR(25) NOT NULL,
    city          VARCHAR(15) NOT NULL,
    postcode      VARCHAR(8),
    type          CHAR(1) NOT NULL
                CHECK (type IN ('B', 'C', 'F', 'M', 'S')) DEFAULT 'F',
    rooms         SMALLINT NOT NULL
                CHECK (rooms BETWEEN 1 AND 15) DEFAULT 4,
    rent          DECIMAL(6,2) NOT NULL
                CHECK (rent BETWEEN 0 AND 9999.99) DEFAULT 600,
    ownerNo       VARCHAR(5) NOT NULL,
    staffNo       VARCHAR(5) CONSTRAINT StaffNotHandlingTooMuch
                CHECK (NOT EXISTS (SELECT staffNo FROM PropertyForRent
                                   GROUP BY staffNo HAVING COUNT(*) > 100)),
    branchNo      CHAR(4) NOT NULL,
    PRIMARY KEY (propertyNo),
    FOREIGN KEY (staffNo) REFERENCES Staff ON DELETE SET NULL
                ON UPDATE CASCADE,
    FOREIGN KEY (ownerNo) REFERENCES PrivateOwner ON DELETE NO ACTION
                ON UPDATE CASCADE,
    FOREIGN KEY (branchNo) REFERENCES Branch ON DELETE NO ACTION
                ON UPDATE CASCADE);

```


Changing a Table Definition

- ❑ The ISO standard provides an **ALTER TABLE** statement for changing the structure of a table once it has been created.

- ❑ The definition of the **ALTER TABLE** statement in the ISO standard consists of six options:
 - add a new column to a table;
 - drop a column from a table;
 - add a new table constraint;
 - drop a table constraint;
 - set a default for a column;
 - drop a default for a column.

Changing a Table Definition

- ❑ The basic format of the statement is:

ALTER TABLE TableName

[**ADD** [**COLUMN**] columnName dataType [**NOT NULL**] [**UNIQUE**]
[**DEFAULT** defaultOption] [**CHECK** (searchCondition)]]

[**DROP** [**COLUMN**] columnName [**RESTRICT** | **CASCADE**]]

[**ADD CONSTRAINT** ConstraintName tableConstraintDefinition]

[**DROP CONSTRAINT** ConstraintName]

[**ALTER** [**COLUMN**] columnName **SET DEFAULT** defaultOption]

[**ALTER** [**COLUMN**] columnName **DROP DEFAULT**]

- The **DROP COLUMN** clause has an optional qualifier that specifies whether the **DROP** action is to cascade or not.
- A *tableConstraintDefinition* is one of the clauses: **PRIMARY KEY**, **UNIQUE**, **FOREIGN KEY**, or **CHECK**.

Changing a Table Definition - Examples

- **ALTER TABLE** Staff
ALTER sex **SET DEFAULT** 'F';
- **ALTER TABLE** Staff
ALTER position **DROP DEFAULT**;
- **ALTER TABLE** PropertyForRent
DROP CONSTRAINT StaffNotHandlingTooMuch;
- **ALTER TABLE** Client
ADD prefNoRooms **SMALLINT NOT NULL** ;

Removing a Table

- ❑ The DROP TABLE statement has the format:

DROP TABLE TableName [**RESTRICT** | **CASCADE**]

- ❑ The DROP TABLE statement allows you to specify whether the DROP action is to be cascaded:
 - **RESTRICT**: The DROP operation is rejected if there are any other objects that depend on the existence of the table to be dropped.
 - **CASCADE**: The DROP operation proceeds and SQL automatically drops all dependent objects (and objects dependent on these objects).
- ❑ The total effect of a DROP TABLE with CASCADE can be very extensive and should be carried out only with extreme caution.
- ❑ Example:
DROP TABLE PropertyForRent;

Creating an Index

- ❑ An **index** is a structure that provides accelerated access to the rows of a table based on the values of one or more columns.
- ❑ Indexes are usually created to satisfy particular search criteria after the table has been in use for some time and has grown in size.

- ❑ The creation of indexes has the format:

```
CREATE [UNIQUE] INDEX IndexName  
ON TableName (columnName [ASC | DESC] [, . . .])
```

- ❑ If the **UNIQUE** clause is used, uniqueness of the indexed column or combination of columns will be enforced by the DBMS.

- ❑ Examples of creating and removing indexes:

- **CREATE UNIQUE INDEX** StaffNoIndex **ON** Staff (staffNo);
- **CREATE INDEX** RentIndex **ON** PropertyForRent (city, rent);
- **DROP INDEX** RentIndex;

Views

- ❑ A **View** is the dynamic result of one or more relational operations operating on the base relations to produce another relation.
- ❑ A view is a *virtual relation* that does not necessarily exist in the database but can be produced upon request by a particular user, at the time of request.

- ❑ A view is defined by specifying an SQL SELECT statement.

```
CREATE VIEW ViewName [(newColumnName [, . . . ])]  
AS subselect
```

- ❑ If a list of column names is specified, it must have the same number of items as the number of columns produced by the *subselect* statement.
- ❑ If the list of column names is omitted, each column in the view takes the name of the corresponding column in the *subselect* statement.

Views

EXAMPLE Create a horizontal view

Create a view so that the manager at branch B003 can see the details only for staff who work in his or her branch office.

```
CREATE VIEW Manager3Staff  
AS SELECT *  
FROM Staff  
WHERE branchNo = 'B003';
```

If we now execute this statement:

```
SELECT * FROM Manager3Staff;
```

we get

staffNo	fName	lName	position	sex	DOB	salary	branchNo
SG37	Ann	Beech	Assistant	F	10-Nov-60	12000.00	B003
SG14	David	Ford	Supervisor	M	24-Mar-58	18000.00	B003
SG5	Susan	Brand	Manager	F	3-Jun-40	24000.00	B003

Views

EXAMPLE **Create a vertical view**

Create a view of the staff details at branch B003 that excludes salary information.

```
CREATE VIEW Staff3  
AS SELECT staffNo, fName, lName, position, sex  
  FROM Staff  
  WHERE branchNo = 'B003';
```

Note that we could rewrite this statement to use the Manager3Staff view instead of the Staff table, thus:

```
CREATE VIEW Staff3  
AS SELECT staffNo, fName, lName, position, sex  
  FROM Manager3Staff;
```

If we list this view, we get

staffNo	fName	lName	position	sex
SG37	Ann	Beech	Assistant	F
SG14	David	Ford	Supervisor	M
SG5	Susan	Brand	Manager	F

Views

EXAMPLE **Grouped and joined views**

Create a view of staff who manage properties for rent, and the number of properties they manage.

```
CREATE VIEW StaffPropCnt (staffNo, cnt)
AS SELECT s.staffNo, COUNT(*)
FROM Staff s, PropertyForRent p
WHERE s.staffNo = p.staffNo
GROUP BY s.staffNo;
```

staffNo	cnt
SG14	1
SG37	2
SL41	1
SA9	1

One of the most frequent reasons for using views is to simplify multi-table queries. Once a joined view has been defined, we can often use a simple single-table query against the view for queries that would otherwise require a multi-table join.

Removing a View

- ❑ A view is removed from the database with the DROP VIEW statement:

DROP VIEW ViewName [**RESTRICT** | **CASCADE**]

- ❑ If **CASCADE** is specified, **DROP VIEW** deletes all related dependent objects. This means that **DROP VIEW** also deletes any views that are defined on the view being dropped.
- ❑ If **RESTRICT** is specified and there are any other objects that depend for their existence on the continued existence of the view being dropped, the command is rejected.
- ❑ For example, we could remove the Manager3Staff view using the following statement:

DROP VIEW Manager3Staff;

Access Control

- ❑ Each user is given appropriate access rights (or *privileges*) on specific database objects.
- ❑ Typically users obtain privileges when they create an object and can pass some or all of these privileges to other users.
- ❑ Each database user is assigned an authorization identifier by the DBA. Usually, the identifier has an associated password, for obvious security reasons.
- ❑ The authorization identifier is used to determine which database objects the user may reference and what operations may be performed on those objects.

Privileges

- ❑ Privileges are the actions that a user is permitted to carry out on a given base table or view.

- ❑ The privileges defined by the ISO standard are
 - SELECT—the privilege to retrieve data from a table;
 - INSERT—the privilege to insert new rows into a table;
 - UPDATE—the privilege to modify rows of data in a table;
 - DELETE—the privilege to delete rows of data from a table;
 - REFERENCES—the privilege to reference columns of a named table in integrity constraints;
 - USAGE—the privilege to use domains.

- ❑ When a user creates a table using the CREATE TABLE statement, he or she automatically becomes the owner of the table and receives full privileges for the table.

Privileges

- ❑ The GRANT statement is used to grant privileges on database objects to specific users. The format of the GRANT statement is:

```
GRANT {PrivilegeList | ALL PRIVILEGES}
ON ObjectName
TO {AuthorizationIdList | PUBLIC}
[WITH GRANT OPTION]
```

- ❑ The **GRANT** statement allows the keyword **ALL PRIVILEGES** to be used to grant all privileges to a user instead of having to specify each privilege individually.
- ❑ The **GRANT** statement also provides the keyword **PUBLIC** to allow access to be granted to all present and future authorized users.
- ❑ The **WITH GRANT OPTION** clause allows the user(s) in *AuthorizationIdList* to pass on the privileges they have been given to other users.

Privileges

EXAMPLE GRANT all privileges

Give the user with authorization identifier Manager all privileges on the Staff table

```
GRANT ALL PRIVILEGES  
ON Staff  
TO Manager WITH GRANT OPTION;
```

The user identified as **Manager** can now retrieve rows from the **Staff** table, and also insert, update, and delete data from this table. **Manager** can also reference the **Staff** table, and all the **Staff** columns in any table that he or she creates subsequently. We also specified the keyword **WITH GRANT OPTION**, so that **Manager** can pass these privileges on to other users.

Privileges

EXAMPLE GRANT specific privileges

Give users Personnel and Director the privileges SELECT and UPDATE on column salary of the Staff table.

```
GRANT SELECT, UPDATE (salary)  
ON Staff  
TO Personnel, Director;
```

We have omitted the keyword **WITH GRANT OPTION**, so that users **Personnel** and **Director** cannot pass either of these privileges on to other users.

Privileges

EXAMPLE GRANT specific privileges to PUBLIC

Give all users the privilege SELECT on the Branch table.

```
GRANT SELECT  
ON Branch  
TO PUBLIC;
```

The use of the keyword **PUBLIC** means that all users (now and in the future) are able to retrieve all the data in the **Branch** table. Note that it does not make sense to use **WITH GRANT OPTION** in this case: as every user has access to the table, there is no need to pass the privilege on to other users.

Revoking Privileges from Users

- ❑ The **REVOKE** statement is used to take away privileges that were granted with the **GRANT** statement.
- ❑ A **REVOKE** statement can take away all or some of the privileges that were previously granted to a user.

- ❑ The format of the statement is:

```
REVOKE [GRANT OPTION FOR] {PrivilegeList | ALL PRIVILEGES}  
ON ObjectName  
FROM {AuthorizationIdList | PUBLIC} [RESTRICT | CASCADE]
```

- ❑ The optional **GRANT OPTION FOR** clause allows privileges passed on via the **WITH GRANT OPTION** of the **GRANT** statement to be revoked separately from the privileges themselves.
- ❑ The **RESTRICT** and **CASCADE** qualifiers operate exactly as in the **DROP TABLE** statement.

Revoking Privileges from Users

EXAMPLE **REVOKE specific privileges from PUBLIC**

Revoke the privilege SELECT on the Branch table from all users.

```
REVOKE SELECT  
ON Branch  
FROM PUBLIC;
```

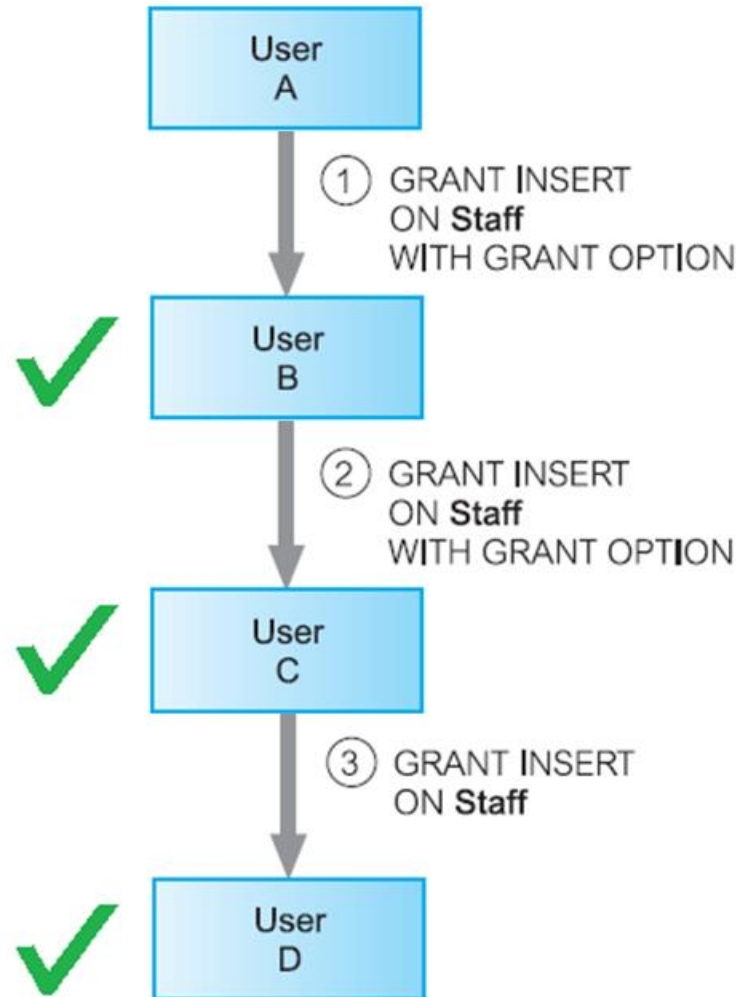
EXAMPLE **REVOKE specific privileges from named user**

Revoke all privileges you have given to Director on the Staff table.

```
REVOKE ALL PRIVILEGES  
ON Staff  
FROM Director;
```

This is equivalent to **REVOKE SELECT, UPDATE ON Staff FROM Director**, as these were the only privileges that have been given to Director.

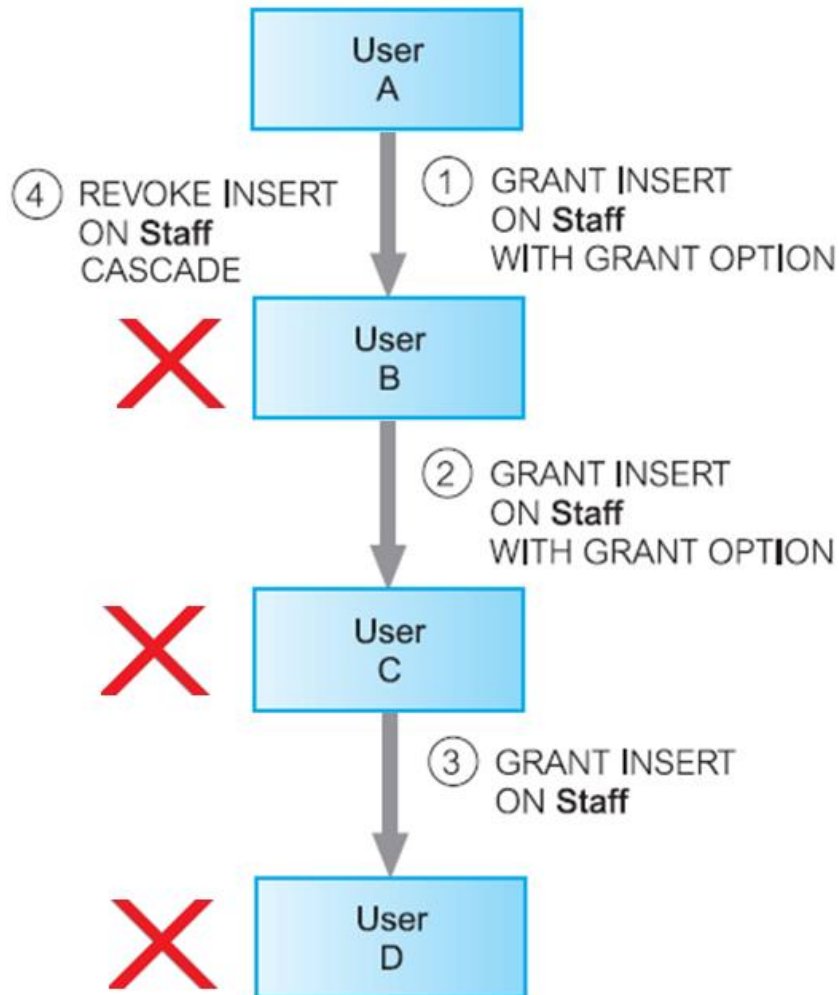
Revoking Privileges from Users – Scenario 1



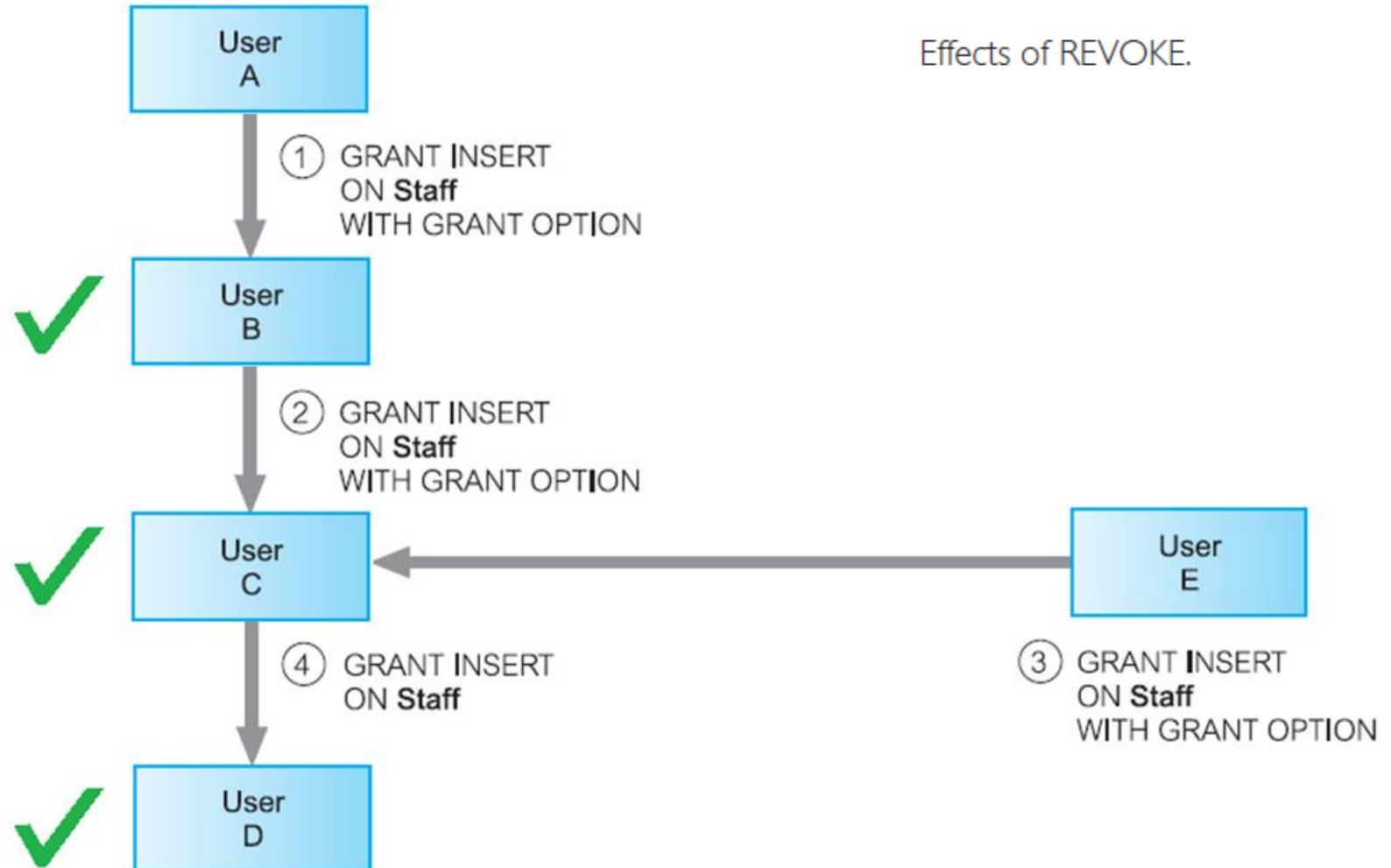
Effects of REVOKE.

Revoking Privileges from Users – Scenario 1

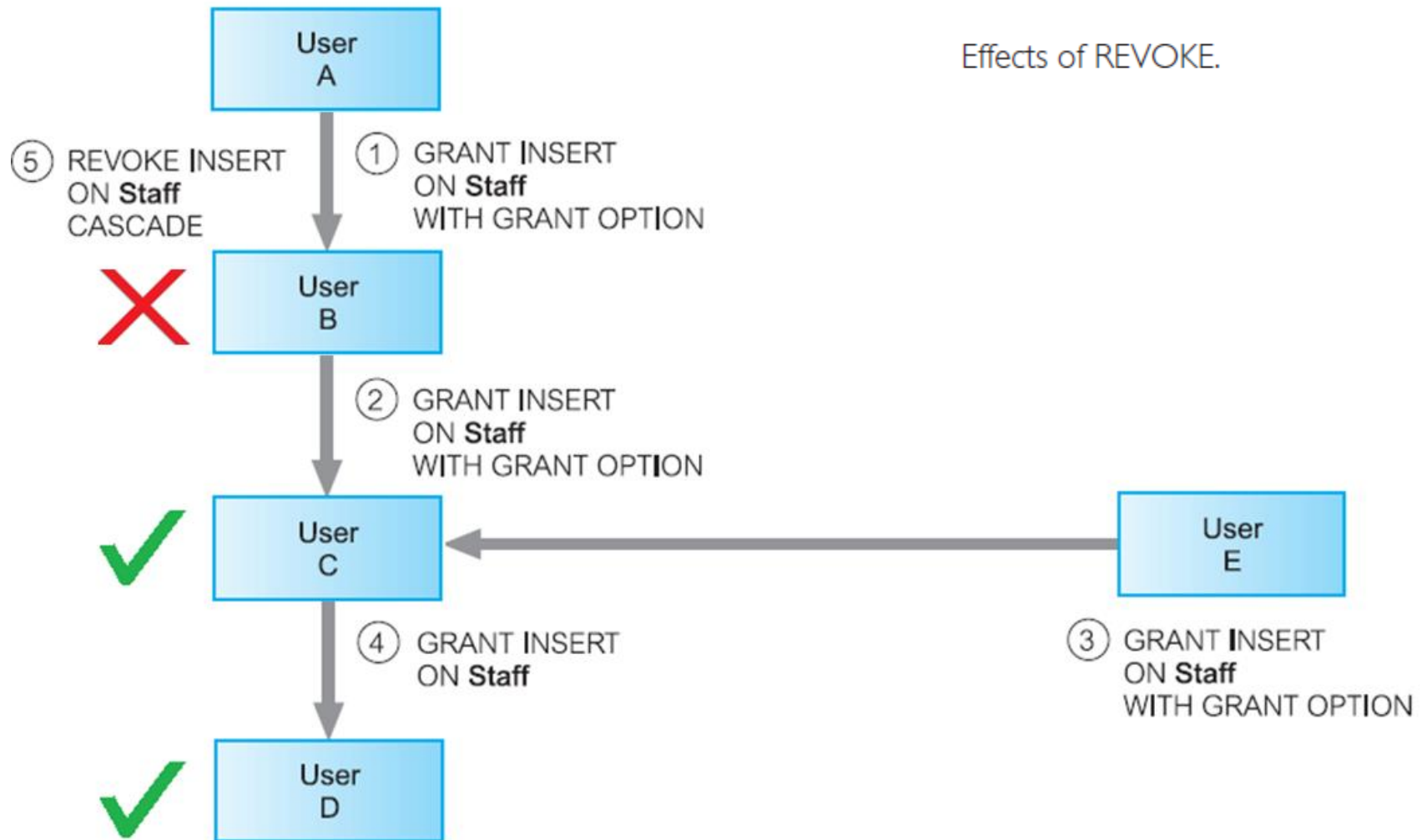
Effects of REVOKE.



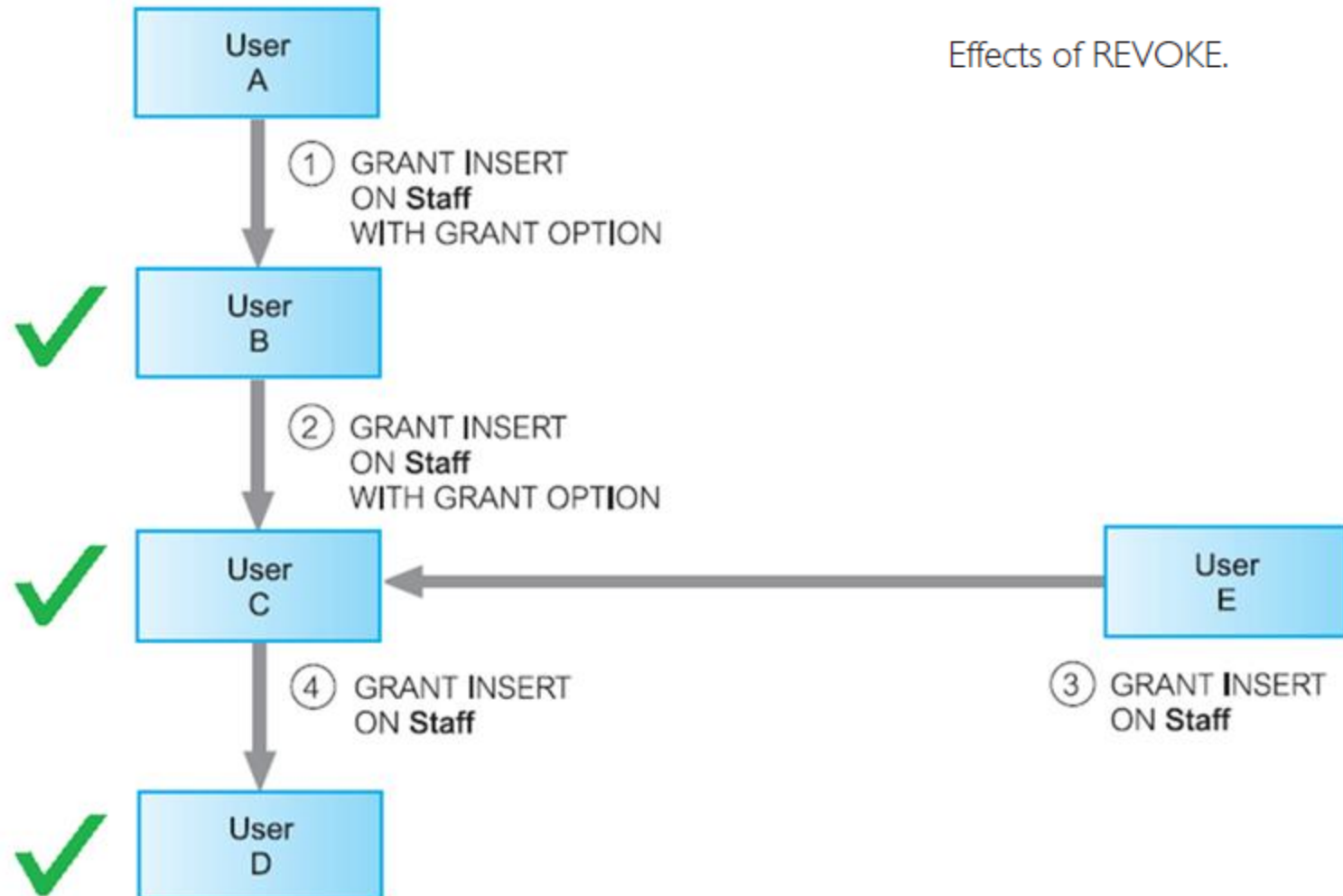
Revoking Privileges from Users – Scenario 2



Revoking Privileges from Users – Scenario 2



Revoking Privileges from Users – Scenario 3



Revoking Privileges from Users – Scenario 3

